

SA Water & Rainfall Intelligence System

ML Engineer Edition — Portfolio Project Breakdown Rainfall Forecasting | Geospatial Dam Mapping | FastAPI Deployment | MLOps

PROJECT OVERVIEW

Build a production-grade forecasting system that predicts South African rainfall 30, 60, and 90 days ahead — then visualises current dam stress levels on an interactive Folium map with the rainfall forecast overlaid. This project mirrors what an ML Engineer does in a real job: shipping working software that combines prediction with geospatial insight.

Field	Detail
Difficulty	Intermediate-Advanced — leverages software engineering background
Total Duration	6 weeks (part-time, ~15 hrs/week)
Problem Type	Time-Series Regression + Geospatial Viz + API Deployment
Impact Domain	Water Security / Agriculture / Climate Intelligence
Target Audience	DWS, municipalities, agricultural sector, general public
Differentiator	Most DS candidates stop at the notebook — you ship a live API with geospatial intelligence

DATA SOURCES

Source	What it provides	Access	Status
Weather SA (rainfall.json)	Daily rainfall, 2000-04-2026 (9,617 entries)	Already scraped	Ready

Source	What it provides	Access	Status
DWS Dam Levels	Current dam storage % for 173 major SA dams	Scraper built	Ready
WorldClim Temperature	Historical min/max temperature rasters	Downloaded	Ready
CHIRPS (bonus)	High-res historical rainfall rasters	Free — optional upgrade	Stretch goal

SKILLS DEMONSTRATED

Pandas · NumPy · GeoPandas · Folium · XGBoost · Time-Series Feature Engineering · sklearn Pipelines · FastAPI · MLflow · Streamlit · Cross-Validation · Matplotlib

FEATURE ENGINEERING

Time-Series Lag Features (Rainfall)

- Rainfall 1, 2, 4, 8, 12 weeks ago (lagged values)
- Rolling 4-week and 12-week average rainfall
- Week-over-week change in rainfall (%)
- Days since last significant rainfall event

Seasonal & Calendar Features

- Month, quarter, season (wet/dry)
- El Nino / La Nina index (if available)
- Days into current dry/wet season
- Year (captures long-term climate trend)

Dam-Specific Features (for geospatial layer)

- Dam storage capacity (normalised)
 - Catchment area size (if available)
 - Primary use: urban supply vs irrigation vs hydropower
 - Historical average level for this week of year
 - Latitude / Longitude (for map visualisation)
-

MODEL APPROACH

Component	Detail
Model Type	XGBRegressor — predicts rainfall at 30/60/90 day horizon
Validation Strategy	TimeSeriesSplit — never use future data to predict the past
Key Metric	MAE in mm rainfall + RMSE for penalty on large errors
Baseline	Naive forecast: "next month = this month" — easy to beat
Multi-horizon	Train 3 separate models: one each for 30, 60, 90 day ahead
Pipeline	sklearn Pipeline: imputer → scaler → XGBoost
Experiment Tracking	MLflow — log MAE, RMSE, params, and model artifacts per run

Critical: Use TimeSeriesSplit NOT random CV — random CV is data leakage

PROJECT STRUCTURE

```
sa-rain-dam-intel/
├── config.py                                ← paths, URLs
├── data/
│   ├── raw/
│   │   ├── rainfall.json                  ← 26 years of
│   │   └── dam_levels/                   ← weekly DWS
│   └── processed/                         ← feature-eng
├── notebooks/
│   └── 01_eda_and_features.ipynb
├── src/
│   ├── scrapers/
│   │   ├── dws_scraper.py                ← Playwright
│   │   └── climate_downloader.py         ← Weather SA
│   ├── features/
│   │   └── engineer.py                   ← lag feature
│   ├── models/
│   │   └── trainer.py                    ← TimeSeriesS
```

└─ viz/	
└─ map_builder.py	← Folium inte
└─ charts.py	← all visuali
└─ models/	← serialised
└─ figures/	← saved chart
└─ requirements.txt	

VISUALISATIONS TO BUILD

Visualisation	Description	Impact
Interactive Dam Map	Folium map — 173 dams coloured red/amber/green by current level %, with rainfall forecast popup	• Hero visual
Rainfall Forecast Line Chart	30/60/90 day predicted rainfall vs historical average per region	• Core
Seasonal Pattern Chart	Average rainfall by month across all 26 years	• Insight
Feature Importance Plot	Which features drive rainfall predictions most?	Interview talking point
Province Stress Summary	Bar chart: which provinces are most water-stressed now	• Stakeholder view
Actual vs Predicted	Time-series plot showing model performance on historical data	• Validation
MLflow Dashboard Screenshot	Shows experiment tracking — engineering maturity	Portfolio

PROJECT TIMELINE — 6 WEEKS

Phase 1 — Week 1 (15 hrs)

Data Preparation & EDA

Day	Task	Deliverable
1	Load rainfall.json into DataFrame, check date range, missing values	Clean rainfall Series
2	Run DWS scraper for all provinces, save CSV	173 dams, current week
3	Build EDA notebook: rainfall distribution, seasonality, rolling stats	EDA notebook
4	Dam data EDA: current levels by province, stress thresholds	Stress summary
5	First pass at feature engineering: lags, rolling windows	Feature notebook

Phase 2 — Week 2 (15 hrs)

Feature Engineering Complete

Day	Task	Deliverable
1	Engineer all lag features (1-12 week lags)	Feature DataFrame
2	Rolling averages (4-week, 12-week)	Feature DataFrame
3	Seasonal features (month, quarter, wet/dry season)	Feature DataFrame
4	Verify no data leakage — check each feature	Clean features
5	Document all features + data leakage risks	Feature doc

Phase 3 — Week 3 (15 hrs)

30-Day Rainfall Model

Day	Task	Deliverable
1	Set up TimeSeriesSplit validation	CV splits
2	Train naive baseline model ("next month = this month")	Baseline MAE
3	Train XGBRegressor for 30-day horizon	

Day	Task	Deliverable
		Trained model
4	Log to MLflow — MAE, RMSE, params, artifacts	MLflow run
5	Plot feature importance + prediction vs actual	Charts

Phase 4 — Week 4 (15 hrs)

60/90-Day Models + Backtest

Day	Task	Deliverable
1-2	Train 60-day and 90-day XGBoost models	3 models total
3	Backtest on historical extreme weather events	Backtest report
4-5	Hyperparameter tuning with early stopping	Optimised models

Phase 5 — Week 5 (15 hrs)

Geospatial Visualisation + API

Day	Task	Deliverable
1	Build Folium interactive dam map with colour coding	Interactive map
2	Add rainfall forecast overlay + popups per dam	Enhanced map
3	Build supporting charts (seasonal, province stress, feature importance)	All charts
4	Build FastAPI <code>/predict</code> endpoint returning rainfall forecast JSON	Live API
5	Generate API docs and test with curl	<code>/docs</code> endpoint

Phase 6 — Week 6 (15 hrs)

Dashboard + Polish + Deploy

Day	Task	Deliverable
1	Build Streamlit dashboard: map + charts + forecast selector	Live dashboard
2	Add weekly data refresh script (cron job)	Automation
3	Add basic drift monitoring (MAE threshold check)	Monitoring
4	Write README with screenshots + architecture diagram	GitHub portfolio
5	Final polish: notebook narrative, interview prep notes	Portfolio ready

WHAT MAKES THIS UNIQUE

This project stands out from typical DS portfolios because:

1. **Geospatial intelligence** — Most weather models are just charts. You put predictions on a real map with 173 actual dams.
2. **Two problems solved at once** — Rainfall forecasting AND water stress monitoring. Shows you think about the full system, not just the model.
3. **Production deployment** — FastAPI + Streamlit + MLflow. Most candidates stop at the notebook.
4. **South African relevance** — Water security is a national issue. This is a real problem, not a Kaggle toy dataset.
5. **Cape Town 2018 backtest (rainfall version)** — Can the model predict drought conditions? Verification against real events.

WHAT TO SAY IN INTERVIEWS

- **Why TimeSeriesSplit not random CV:** Random CV uses future data to predict the past — data leakage
- **Why 3 separate models:** Each horizon captures different lag relationships — one model cannot serve all three
- **Why rainfall + dam map together:** Pure rainfall data is abstract — putting it on a map with real dam levels makes it actionable for decision-makers
- **Why FastAPI over Flask:** Async support, automatic docs at / docs, faster for ML inference endpoints

- **Why MLflow:** Without experiment tracking you cannot reproduce results — essential on any real ML team
 - **Real-world limitation:** This model predicts rainfall, not dam inflows directly — evaporation, usage, and releases also affect levels
-

STRETCH GOALS

- Docker container: Containerise the FastAPI app — deploy to any cloud with one command
 - Drought early warning: Auto-alert when any dam is forecast to drop below 30% within 60 days
 - Scenario simulator: Streamlit slider — "what if rainfall drops 50%?" — re-run forecast live
 - CI/CD pipeline: GitHub Actions — auto-run tests and redeploy API on every code push
 - Model retraining trigger: Auto-retrain when drift is detected — fully automated ML lifecycle
-

Portfolio Project Breakdown • SA Water & Rainfall Intelligence System
• 6-Week Sprint